

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 7

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Linux Process Address Space Layout

Question:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week7$ cat > P7_T1.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;
static int static_var = 200;

void code_function(void)
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main(void)
{
    int stack_var = 300;

    int *heap_var = malloc(sizeof(int));

    if (heap_var == NULL)
    {
        perror("malloc");
        return 1;
    }

    *heap_var = 400;

    printf("Process PID      : %d\n\n", getpid());

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    printf("\nProcess is paused for memory analysis.\n");
    printf("Press ENTER to terminate...\n");

    getchar();

EOF return 0;_var);
```

Output:

```
haasini@SSH:~/OSSP_upload/Practical_Week7$ ./P7_T1
Process PID      : 569

Code address     : 0x597b320d3229
Global address   : 0x597b320d6010
Static address   : 0x597b320d6014
Heap address     : 0x597b3c34c2a0
Stack address    : 0x7ffc27730cac

Process is paused for memory analysis.
Press ENTER to terminate...

haasini@SSH:~/OSSP_upload/Practical_Week7$ cat /proc/569/maps
cat: /proc/569/maps: No such file or directory
haasini@SSH:~/OSSP_upload/Practical_Week7$
```

Observations:

1. The code/function address (0x597b320d3229) belongs to the executable code/text region.
2. Initialized global and static variables are stored in the program's data area (0x597b320d6010 and 0x597b320d6014 respectively).
3. Dynamically allocated memory from malloc() is obtained from the heap region (0x597b3c34c2a0).
4. The local automatic variable is stored on the stack (0x7ffc27730cac).
5. Different regions appear at different virtual addresses in the process memory layout.
6. The displayed addresses are virtual addresses in the process's virtual address space.
7. The exact addresses vary between executions because of mechanisms such as ASLR (Address Space Layout Randomization).
8. The program correctly demonstrates all five key memory segments: code, global, static, heap, and stack.
9. The PID 569 identifies the unique running process instance.
10. The process pauses to allow inspection of memory addresses during execution.

Task 2: Virtual Memory Mapping Analysis Using /proc and pmap

Question:

Make use of the /proc/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Output:

/proc/[PID]/maps Output (Filtered)

```
haasini@SSH:~/OSSP_upload/Practical_Week7$ grep -E 'P7_T1|\\[heap\\]|\\[stack\\]' /proc/569/maps
597b320d2000-597b320d3000 r--p 00000000 08:30 11524 /home/haasini/OSSP_upload/Practical_Week7/P7_T
597b320d3000-597b320d4000 r-xp 00001000 08:30 11524 /home/haasini/OSSP_upload/Practical_Week7/P7_T
597b320d4000-597b320d5000 r--p 00002000 08:30 11524 /home/haasini/OSSP_upload/Practical_Week7/P7_T
597b320d5000-597b320d6000 r--p 00002000 08:30 11524 /home/haasini/OSSP_upload/Practical_Week7/P7_T
597b320d6000-597b320d7000 rw-p 00003000 08:30 11524 /home/haasini/OSSP_upload/Practical_Week7/P7_T
597b3c34c000-597b3c36d000 rw-p 00000000 00:00 0 [heap]
7ffc27711000-7ffc27732000 rw-p 00000000 00:00 0 [stack]
```

pmap -x [PID] Output

```
total 2688K
haasini@SSH:~/OSSP_upload/Practical_Week7$ pmap -x 569
569:  . /P7_T1
Address          Kbytes      RSS    Dirty Mode  Mapping
0000597b320d2000      4         4        0 r---- P7_T1
0000597b320d3000      4         4        0 r-x-- P7_T1
0000597b320d4000      4         4        0 r---- P7_T1
0000597b320d5000      4         4        4 r---- P7_T1
0000597b320d6000      4         4        4 rw--- P7_T1
0000597b3c34c000    132         4        4 rw--- [ anon ]
000076b4e9600000    160        156        0 r---- libc.so.6
000076b4e9628000   1568        992        0 r-x-- libc.so.6
000076b4e97b0000    316         64        0 r---- libc.so.6
000076b4e97ff000     16         16       16 r---- libc.so.6
000076b4e9803000      8          8        8 rw--- libc.so.6
000076b4e9805000     52         20       20 rw--- [ anon ]
000076b4e9957000     12          8        8 rw--- [ anon ]
000076b4e995f000      8          4        4 rw--- [ anon ]
000076b4e9961000     16          0        0 r---- [ anon ]
000076b4e9965000      8          0        0 r---- [ anon ]
000076b4e9967000      8          4        0 r-x-- [ anon ]
000076b4e9969000      4          4        0 r---- ld-linux-x86-64.so.2
000076b4e996a000    172        172        0 r-x-- ld-linux-x86-64.so.2
000076b4e9995000     40         40        0 r---- ld-linux-x86-64.so.2
000076b4e999f000      8          8        8 r---- ld-linux-x86-64.so.2
000076b4e99a1000      8          8        8 rw--- ld-linux-x86-64.so.2
00007ffc27711000    132         12       12 rw--- [ stack ]
-----
total kB          2688    1540     96
```

Observations:

1. /proc/[PID]/maps describes the virtual memory mappings of the running process (PID 569).
2. Mappings contain virtual address ranges specifying the start and end addresses of each region.
3. Permissions such as r (read), w (write), x (execute), and p (private) describe access properties of each mapping.
4. The executable P7_T1 appears in multiple mapped regions with different permissions (code, initialized data, uninitialized data).
5. [heap] identifies the process heap region where dynamic memory allocations are stored.
6. [stack] identifies the process stack region where local variables and function parameters reside.
7. Shared libraries such as libc.so.6 appear as separate mappings with their own address ranges and permissions.
8. pmap provides a more readable summary of the process memory mappings compared to raw /proc/maps output.
9. pmap -x provides additional memory usage information such as mapping size (Kbytes), resident memory (RSS), and dirty pages.
10. /proc/[PID] exists while that process exists and is removed when the process terminates.